

EE DOCKNER

Complete Interview Preparation Guide

Intermediate SRE / Cloud Operations Engineer (4 Years)

DevOps Engineer • Platform Engineer • Infrastructure Engineer

■ 100+ Interview Questions ■ Production Scenarios ■ Troubleshooting Deep Dives
■ AWS Integration ■ Linux Commands ■ CI/CD Pipelines

Interview-First • Production-Focused • AWS-Aligned • SRE Perspective

Study Priority Guide

Module	Topic	Pages	Interview Weight
1	Docker Fundamentals & Architecture	~8	■■■
2	Images, Containers & Dockerfile	~10	■■■■■
3	Networking & Storage (Volumes)	~10	■■■■■
4	Docker Compose	~8	■■■■■
5	Docker Commands (Deep Dive)	~8	■■■■■
6	Security & Best Practices	~8	■■■■
7	Docker in CI/CD & AWS	~8	■■■■
8	Production Troubleshooting	~10	■■■■■
9	Top 100 Q&A; + Cheat Sheets	~10	■■■■■

■ Priority: Master Modules 2, 3, 4, 5, 8 — these cover 80% of Docker interview questions for SRE/DevOps roles.

MODULE 1 — Docker Fundamentals & Architecture

■■■ Priority

1.1 What is Docker?

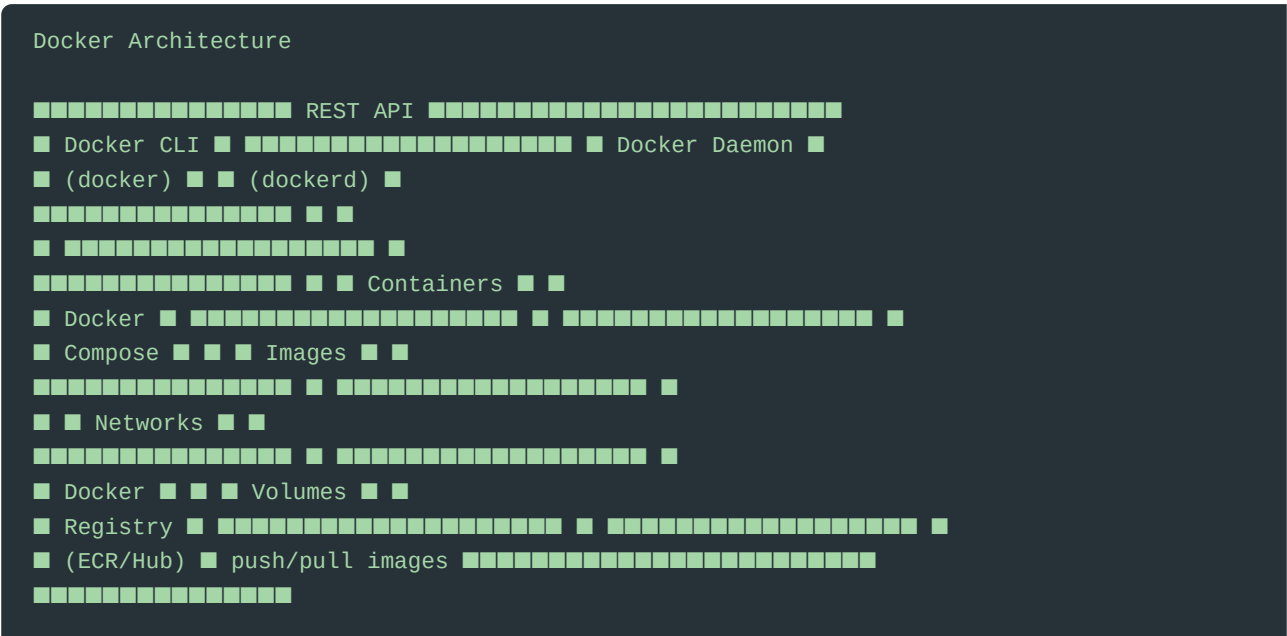
Docker is an open-source platform that packages applications and their dependencies into **containers** — isolated, lightweight, portable units that run consistently across any environment. Docker solves the classic "it works on my machine" problem.

Why Docker Exists

Problem (Before Docker)	Solution (With Docker)
Application works on dev, breaks on prod	Container packages app + all dependencies
Slow, heavy VMs for each service	Lightweight containers share OS kernel
Inconsistent environments across teams	Same image runs everywhere
Complex dependency management	Dependency isolation per container
Hard to scale individual services	Containers start in seconds, easy to replicate

1.2 Docker Architecture

Docker uses a **client-server architecture**. The Docker Client communicates with the Docker Daemon (dockerd) via REST API. The Daemon manages images, containers, networks, and volumes.



Key Components

Component	What It Does
Docker Daemon (dockerd)	Background service that manages Docker objects (containers, images, volumes, networks)
Docker CLI (docker)	Command-line interface to interact with the daemon

Docker Registry	Stores and distributes Docker images (Docker Hub, AWS ECR, private registries)
Docker Images	Read-only templates used to create containers
Docker Containers	Running instances of Docker images
Docker Networks	Virtual networks connecting containers
Docker Volumes	Persistent storage for containers

1.3 Docker vs Virtual Machines

Aspect	Docker Container	Virtual Machine
OS	Shares host OS kernel	Full OS per VM
Size	MBs	GBs
Startup	Seconds	Minutes
Isolation	Process-level (namespaces)	Full hardware isolation
Overhead	Very low	High
Portability	High	Medium
Use Case	Microservices, CI/CD, cloud-native	Full OS requirements, legacy apps

■■■■■ Interview Q: How does Docker achieve isolation without a hypervisor?

Answer: Docker uses Linux kernel features: **Namespaces** (pid, net, mnt, uts, ipc, user) for isolation and **cgroups** (control groups) for resource limiting (CPU, memory). Containers share the host kernel but have isolated process trees and filesystems.


```
# ■■ Stage 1: Build ████████████████████████████████████████████████████████████  
FROM python:3.11-slim AS builder  
  
WORKDIR /app  
  
# Copy dependency files FIRST (cache optimization)  
COPY requirements.txt .  
RUN pip install --no-cache-dir --user -r requirements.txt  
  
# ■■ Stage 2: Production ████████████████████████████████████████████████████████████  
FROM python:3.11-slim  
  
# Create non-root user (security best practice)  
RUN groupadd -r appuser && useradd -r -g appuser appuser  
  
WORKDIR /app  
  
# Copy installed packages from builder stage  
COPY --from=builder /root/.local /root/.local  
  
# Copy application code  
COPY --chown=appuser:appuser . .  
  
# Set environment  
ENV PATH=/root/.local/bin:$PATH  
ENV PYTHONDONTWRITEBYTECODE=1  
ENV PYTHONUNBUFFERED=1  
  
# Switch to non-root user  
USER appuser  
  
# Document the port (does NOT expose - use -p at runtime)  
EXPOSE 8000  
  
# Healthcheck  
HEALTHCHECK --interval=30s --timeout=10s --start-period=5s --retries=3 \  
CMD curl -f http://localhost:8000/health || exit 1  
  
# Entry point  
CMD ["python", "-m", "uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Key Dockerfile Instructions

Instruction	Purpose	Interview Note
FROM	Base image — must be first instruction	★★★★★ Always asked
RUN	Execute commands during build (creates layer)	★★★★★ Layer caching questions
COPY	Copy files from host to image	★★★★■ COPY vs ADD
ADD	Like COPY but can extract archives, fetch URLs	★★★★■ Use COPY unless you need ADD features
ENV	Set environment variables (persists in container)	★★★★■ Visible in image inspect

ARG	Build-time variable only (not in final image)	★★★★■ ARG vs ENV difference
WORKDIR	Set working directory (creates if missing)	★★★★■ Best practice over cd
EXPOSE	Document which port app uses (informational)	★★★★■ Does NOT publish port
CMD	Default command — overridable at runtime	★★★★★ CMD vs ENTRYPOINT
ENTRYPOINT	Fixed command — CMD becomes its arguments	★★★★★ CMD vs ENTRYPOINT
HEALTHCHECK	Tell Docker how to check container health	★★★★■ Production essential
USER	Switch to non-root user (security)	★★★★★ Security interviews
VOLUME	Declare mount point (creates anonymous volume)	★★★★■ Use named volumes instead
LABEL	Add metadata to image	★★★★■ Often used for CI metadata
ONBUILD	Trigger instructions in child images	★★■■■ Rarely asked

CMD vs ENTRYPOINT

This is one of the most frequently asked Dockerfile questions in interviews.

```
# CMD – default command, easily overridden
CMD ["python", "app.py"]
# docker run myimage python other.py ← overrides CMD completely

# ENTRYPOINT – fixed command, CMD becomes arguments
ENTRYPOINT ["python"]
CMD ["app.py"]
# docker run myimage other.py ← runs: python other.py
# docker run myimage ← runs: python app.py

# Together (best pattern for production):
ENTRYPOINT ["python", "app.py"]
CMD ["--port", "8000"] ← default argument
```

	CMD	ENTRYPOINT
Override at runtime	Yes, easily (just add command)	Requires --entrypoint flag
Use case	Default args / default command	Fixed executable, configurable args
Shell form	CMD python app.py	ENTRYPOINT python app.py
Exec form (preferred)	CMD ["python", "app.py"]	ENTRYPOINT ["python", "app.py"]
Signal handling	Shell form: signals NOT passed to app	Exec form: signals passed correctly

2.3 Multi-Stage Builds

Multi-stage builds allow you to use multiple FROM instructions in one Dockerfile. The final image only contains what you explicitly copy from earlier stages — dramatically reducing image size and attack surface.

Interview Q: Why use multi-stage builds?

Answer: To separate build tools from the runtime image. For example, compiling a Go binary needs a full Go SDK (1GB+), but the final binary can run in a scratch or alpine image (5MB). Result: smaller image, faster pull, smaller attack surface, better security.

```
# Go example: build stage vs final stage
FROM golang:1.21 AS builder # Full build env (~1GB)
WORKDIR /app
COPY . .
RUN go build -o myapp .

FROM scratch # Empty base image (~0MB)
COPY --from=builder /app/myapp /myapp
CMD ["/myapp"]

# Final image: only your binary — no Go SDK, no shell, no extra packages
```

Layer Caching — Cache Optimization (Critical for CI/CD)

```
# BAD - invalidates cache on ANY code change
COPY . .
RUN pip install -r requirements.txt

# GOOD - dependencies cached separately from code
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . . ← only code changes here; deps layer stays cached

Rule: Order Dockerfile instructions from LEAST to MOST frequently changing.
```

2.4 Container Lifecycle

Container States:

```
Created → Running → Paused → Running → Stopped → Removed
```

Exited docker start

State	Description	Command
Created	Container created but not started	docker create
Running	Container actively executing	docker start / docker run
Paused	Container processes frozen (SIGSTOP)	docker pause
Stopped/Exited	Container stopped (can restart)	docker stop
Removed	Container deleted permanently	docker rm

■■■■■ Priority

Docker networking enables containers to communicate with each other and with the outside world. Docker provides several built-in network drivers.

Bridge Network — How It Works

[illegible]

Answer: Docker provides automatic DNS resolution on user-defined bridge networks. Containers can reach each other by **container name** (e.g., `http://db:5432`). This does NOT work on the default bridge network — you must use user-defined networks. Use `docker network create mynet` and add containers to it.

Page 9

```
# -p hostPort:containerPort
docker run -p 80:8080 myapp # Host port 80 → Container port 8080
docker run -p 443:443 nginx # HTTPS
docker run -P myapp # Auto-map all EXPOSE'd ports to random host ports

# View port mappings:
docker port
docker inspect | grep -A 10 Ports
```

3.2 Docker Volumes (Persistent Storage)

Container filesystems are ephemeral — data is lost when a container is removed. Volumes provide persistent storage that exists independently of container lifecycle.

Storage Type	Description	When to Use
Named Volume	Docker-managed storage in <code>/var/lib/docker/volumes/</code> . Portable, recommended.	Databases, persistent app data
Bind Mount	Maps a host directory into container. Host path must exist.	Dev environments, config files, sharing code
tmpfs Mount	In-memory storage. Lost on container stop.	Secrets, sensitive temp data, performance
Anonymous Volume	Like named volume but auto-named. Hard to manage.	Avoid in production — use named volumes

```
# Named Volume (Recommended for production)
docker volume create mydata
docker run -v mydata:/var/lib/postgresql/data postgres

# Bind Mount (Dev workflow)
docker run -v /home/user/app:/app myimage # absolute path
docker run -v $(pwd):/app myimage # current directory

# Read-only bind mount
docker run -v /config:/etc/app:ro myimage

# tmpfs (in-memory, no persistence)
docker run --tmpfs /tmp:size=100m myimage

# Inspect volumes
docker volume ls
docker volume inspect mydata
docker volume rm mydata
```

■■■■■ Interview Q: Volumes vs Bind Mounts — when to use which?

Named Volumes: Use in production. Docker manages location, portable between containers, works with docker-compose, can use volume drivers for cloud storage (EFS, EBS).

Bind Mounts: Use in development. Direct host filesystem access, great for live code reload, but ties you to host path structure and can cause permission issues.

Volume in Compose (Production Pattern)

```
services:
  postgres:
    image: postgres:15
    volumes:
      - pgdata:/var/lib/postgresql/data # Named volume
    environment:
      POSTGRES_PASSWORD_FILE: /run/secrets/pg_pass

volumes:
  pgdata: # Declare at top level – persists across docker-compose down/up
```

4.1 What is Docker Compose?

Docker Compose is a tool for defining and running **multi-container applications** using a YAML file. It orchestrates startup order, networking, volumes, and environment configuration in one command.

Production-Quality docker-compose.yml

```
version: "3.9"
services:
  web:
    build: { context: ., dockerfile: Dockerfile, target: production }
    image: myapp:${TAG:-latest}
    restart: unless-stopped
    ports: ["8000:8000"]
    environment:
      - DATABASE_URL=postgresql://user:pass@db:5432/mydb
    env_file: [.env]
    depends_on:
      db:
        condition: service_healthy
    networks: [backend]
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost:8000/health"]
      interval: 30s
      timeout: 10s
      retries: 3
      start_period: 15s
    deploy:
      resources:
        limits: { cpus: "0.5", memory: 512M }
```

```

db:
  image: postgres:15-alpine
  restart: unless-stopped
  volumes: [pgdata:/var/lib/postgresql/data]
  environment:
    POSTGRES_DB: mydb
    POSTGRES_USER: user
    POSTGRES_PASSWORD: pass
  networks: [backend]
  healthcheck:
    test: ["CMD-SHELL", "pg_isready -U user -d mydb"]
    interval: 10s
    timeout: 5s
    retries: 5

redis:
  image: redis:7-alpine
  restart: unless-stopped
  command: redis-server --requirepass ${REDIS_PASSWORD}
  volumes: [redisdata:/data]
  networks: [backend]

networks:
  backend:
    driver: bridge

volumes:
  pgdata:
  redisdata:

```

Key Compose Concepts

Concept	Explanation
depends_on	Controls startup order. With condition: service_healthy, waits for healthcheck to pass.
restart policy	no always on-failure unless-stopped. Use unless-stopped in production.
networks	Services on same network communicate by service name (built-in DNS).
env_file	Load variables from a .env file. Keep secrets out of compose file.
profiles	Start subset of services: docker compose --profile debug up
healthcheck	Define how Docker checks if service is ready. Critical for depends_on conditions.

Essential Compose Commands

```
# Start services (build if needed)
docker compose up -d # detached (background)
docker compose up --build -d # force rebuild

# Stop services
docker compose down # stop + remove containers + networks
docker compose down -v # also remove volumes (careful in prod!)
docker compose stop # stop but keep containers

# Logs
docker compose logs -f # follow all logs
docker compose logs -f web # follow specific service

# Scaling
docker compose up -d --scale web=3 # run 3 web containers

# Exec into running container
docker compose exec web bash

# Check status
docker compose ps
docker compose top
```

■■■■■ Interview Q: How do you ensure the database is ready before the app starts?

Answer: Use `depends_on` with condition: `service_healthy` and define a `HEALTHCHECK` on the DB service. Without health conditions, `depends_on` only waits for the container to `START`, not for the service inside to be `READY`. For postgres: `CMD-SHELL: pg_isready -U user`. Applications should also implement retry logic — never assume DB is always immediately available.

6.1 Container Security Principles

Security Area	Best Practice	Implementation
Non-root user	Never run containers as root	USER appuser in Dockerfile
Read-only filesystem	Prevent filesystem writes	docker run --read-only
Minimal base image	Reduce attack surface	Use alpine, distroless, or scratch
No secrets in images	Secrets must not be baked into images	Use env vars, secrets, or Secrets Manager
Scan images	Find vulnerabilities before deployment	docker scout, trivy, AWS ECR scanning
Resource limits	Prevent container from consuming all resources	--memory, --cpus flags
No privileged mode	Avoid --privileged flag	Use specific capabilities instead
Network segmentation	Isolate containers on separate networks	User-defined networks per service tier

6.2 Secrets Management

■■ NEVER put secrets in ENV variables in Dockerfile or bake them into images. They appear in docker inspect, docker history, and container logs.

```
# BAD – Secret visible in image history and env
ENV DB_PASSWORD=mysecret123

# GOOD Options:

# Option 1: Runtime env var (not in image)
docker run -e DB_PASSWORD=$SECRET myimage

# Option 2: Docker Secrets (Swarm)
echo "mysecret" | docker secret create db_password -
# In compose:
secrets:
  db_password:
    external: true
services:
  app:
    secrets:
      - db_password # mounted at /run/secrets/db_password

# Option 3: AWS Secrets Manager (production)
# App reads secret at runtime via boto3/SDK – never in image
```

6.3 Dockerfile Security Checklist

- Use specific base image tags (not :latest)
- Pin dependency versions
- Create and use non-root user
- Use multi-stage builds to exclude build tools
- Minimize layers with &&
- Clean up package manager cache in same RUN instruction
- Set HEALTHCHECK
- Use .dockerignore to exclude sensitive files
- Scan images with trivy or docker scout before pushing

```
# .dockerignore example (MUST have for every project)
.git
.gitignore
.env
.env.*
*.pyc
__pycache__/
node_modules/
.pytest_cache/
*.log
README.md
tests/
.terraform/
*.tfstate
```

6.4 Resource Limits

```
# CPU & Memory limits
docker run --memory=512m --memory-swap=512m myimage # 512MB RAM, no swap
docker run --cpus=0.5 myimage # Half a CPU core
docker run --cpu-shares=512 myimage # Relative CPU priority

# In Compose:
deploy:
  resources:
    limits:
      cpus: "0.5"
      memory: 512M
    reservations: # Guaranteed minimum
      cpus: "0.25"
      memory: 256M
```

7.1 Docker in GitHub Actions

```
name: Build and Push to ECR

on:
  push:
    branches: [main]

env:
  AWS_REGION: us-east-1
  ECR_REPOSITORY: myapp

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Configure AWS credentials
        uses: aws-actions/configure-aws-credentials@v4
        with:
          aws-access-key-id: ${ secrets.AWS_ACCESS_KEY_ID }
          aws-secret-access-key: ${ secrets.AWS_SECRET_ACCESS_KEY }
          aws-region: ${ env.AWS_REGION }

      - name: Login to Amazon ECR
        id: login-ecr
        uses: aws-actions/amazon-ecr-login@v2

      - name: Build, tag, and push image
        env:
          ECR_REGISTRY: ${ steps.login-ecr.outputs.registry }
          IMAGE_TAG: ${ github.sha }
        run: |
          docker build -t $ECR_REGISTRY/$ECR_REPOSITORY:$IMAGE_TAG .
          docker push $ECR_REGISTRY/$ECR_REPOSITORY:$IMAGE_TAG
          # Also tag as latest
          docker tag $ECR_REGISTRY/$ECR_REPOSITORY:$IMAGE_TAG \
            $ECR_REGISTRY/$ECR_REPOSITORY:latest
          docker push $ECR_REGISTRY/$ECR_REPOSITORY:latest

      - name: Scan image for vulnerabilities
        run: |
          docker pull aquasec/trivy:latest
          docker run --rm -v /var/run/docker.sock:/var/run/docker.sock \
            aquasec/trivy image --exit-code 1 --severity HIGH,CRITICAL \
            $ECR_REGISTRY/$ECR_REPOSITORY:$IMAGE_TAG
```

7.2 Docker on AWS

AWS Service	Docker Use Case	Key Points
Amazon ECR	Private Docker registry	IAM-based auth, image scanning, lifecycle policies, replication
Amazon ECS	Run containers (managed)	Task definitions = docker run; Services = replicas; Fargate = serverless
Amazon EKS	Kubernetes (K8s) on AWS	Pods run containers; use ECR images; IRSA for IAM
AWS App Runner	Container to URL in minutes	Auto-scales from ECR; great for APIs
AWS Batch	Batch container jobs	Docker containers for data processing jobs
CodeBuild	Build Docker images in CI	Supports buildspec.yml with docker commands

ECS Task Definition (equivalent to docker run)

```
{
  "family": "myapp",
  "containerDefinitions": [{
    "name": "web",
    "image": "123456789012.dkr.ecr.us-east-1.amazonaws.com/myapp:latest",
    "memory": 512,
    "cpu": 256,
    "portMappings": [{"containerPort": 8000, "protocol": "tcp"}],
    "environment": [{"name": "ENV", "value": "production"}],
    "secrets": [{
      "name": "DB_PASSWORD",
      "valueFrom": "arn:aws:secretsmanager:us-east-1:123456789:secret:db-pass"
    }],
    "logConfiguration": {
      "logDriver": "awslogs",
      "options": {
        "awslogs-group": "/ecs/myapp",
        "awslogs-region": "us-east-1",
        "awslogs-stream-prefix": "ecs"
      }
    },
    "healthCheck": {
      "command": ["CMD-SHELL", "curl -f http://localhost:8000/health || exit 1"],
      "interval": 30, "timeout": 5, "retries": 3
    }
  ]},
  "requiresCompatibilities": ["FARGATE"],
  "networkMode": "awsvpc",
  "cpu": "256", "memory": "512"
}
```

8.1 General Troubleshooting Approach

Production Debugging Flow:

1. Check container status
`docker ps -a`
`docker inspect`
2. Check logs
`docker logs --tail 200 -f`
3. Check resource usage
`docker stats`
4. Exec into container
`docker exec -it bash (or sh for alpine)`
5. Check networking
`docker network inspect`
`docker exec ping`
`docker exec curl http://service:port`
6. Check volumes
`docker inspect | grep -A 20 Mounts`

8.2 Common Production Scenarios

■ Scenario: Container exits immediately after start

1. `docker logs` – check error messages
2. `docker inspect | grep ExitCode`
3. Common causes: missing env var, port conflict, entrypoint error, missing file
4. Test: `docker run --entrypoint /bin/sh myimage (override entrypoint)`

■ Scenario: Container uses too much memory (OOMKilled)

1. `docker inspect | grep OOMKilled` → true means killed by kernel
2. `docker stats` → watch memory usage in real time
3. Fix: increase memory limit or optimize application
4. Linux: `dmesg | grep -i "out of memory"` – kernel OOM logs

■ Scenario: Cannot connect to container service

1. Check if container is running: `docker ps`
2. Check port mapping: `docker port`
3. Check container listening: `docker exec netstat -tlnp`
4. Test from host: `curl localhost:`
5. Test between containers: `docker exec curl http://service:port`
6. Check network: `docker network inspect`

■ Scenario: Disk space full — /var/lib/docker

1. Check disk: `df -h /var/lib/docker`
2. Find large objects: `docker system df`
3. Clean up safely: `docker system prune` (removes stopped containers, dangling images)
4. Nuclear: `docker system prune -a --volumes` (removes ALL unused)
5. Production: set up ECR lifecycle policies, limit logs with `--log-opt max-size`

■ Scenario: Docker build failing / slow build

1. No-cache test: `docker build --no-cache -t test .`
2. Check `.dockerignore` – large directories slow COPY
3. Layer cache invalidated: reorder COPY instructions (deps first, code last)
4. Network issue during build: check DNS, proxy settings inside build

■ Scenario: Container starts but healthcheck fails

1. `docker inspect | grep -A 10 Health`
2. `docker events --watch health status transitions`
3. Test healthcheck manually: `docker exec curl http://localhost/health`
4. Increase `start_period` if app needs longer to initialize
5. Check logs: `docker logs`

8.3 Linux Commands for Docker Troubleshooting

```
# Process & Resource
ps aux | grep docker
top -p $(pgrep dockerd)
systemctl status docker
journalctl -u docker -f # Docker daemon logs

# Disk & Storage
df -h /var/lib/docker
du -sh /var/lib/docker/*
ls -lh /var/lib/docker/volumes/

# Networking
ss -tlnp | grep docker
iptables -L -n -v | grep DOCKER # Docker iptables rules
ip addr show docker0 # Bridge interface
ip route # Routing table

# cgroups (resource inspection)
cat /sys/fs/cgroup/memory/docker//memory.usage_in_bytes

# Linux kernel namespaces (containers)
lsns | grep # List namespaces
nsenter -t -n ip addr # Enter container's network namespace
```

MODULE 9 — Top Interview Questions & Answers

■■■■■ Priority

Top 60 Docker Interview Questions

(★★★★★ = asked in almost every interview | ★★★★ = frequently asked | ★★★ = sometimes asked)

★★★★★ What is Docker and why is it used?

Docker is a containerization platform that packages applications and all dependencies into portable containers. Used because containers run consistently across any environment, start in seconds, use minimal resources, and enable microservices architecture.

★★★★★ What is the difference between a Docker image and a Docker container?

An image is a read-only template with app code and dependencies. A container is a running instance of an image — it adds a writable layer on top. Multiple containers can run from the same image simultaneously.

★★★★★ Explain Docker architecture.

Client-server architecture. Docker CLI (client) sends commands via REST API to Docker Daemon (dockerd). The daemon manages images, containers, networks, and volumes. Images are stored in registries (Docker Hub, ECR).

★★★★★ How does Docker achieve isolation?

Docker uses Linux kernel features: Namespaces (pid, net, mnt, uts, ipc, user) for process/resource isolation, and cgroups for resource limiting (CPU, memory, I/O). Containers share the host kernel but have isolated views.

★★★★★ What is a Dockerfile? What are the most important instructions?

A Dockerfile is a text file with build instructions. Key instructions: FROM (base image), RUN (execute commands), COPY (add files), ENV (environment variables), EXPOSE (document port), CMD/ENTRYPOINT (start command), USER (non-root), HEALTHCHECK, WORKDIR.

★★★★★ Difference between CMD and ENTRYPOINT?

ENTRYPOINT defines the fixed executable that always runs. CMD provides default arguments to ENTRYPOINT or a default command if no ENTRYPOINT. CMD is easily overridden at runtime; ENTRYPOINT requires --entrypoint flag. Best practice: ENTRYPOINT for the executable, CMD for default args.

★★★★★ What are Docker image layers and how does caching work?

Each Dockerfile instruction creates an immutable layer. During builds, Docker checks if the instruction + its context has changed. If not, it uses the cached layer. Caching is invalidated if any layer above changes. Order instructions from least-to-most frequently changing for optimal cache reuse.

★★★★★ What is a multi-stage build and why use it?

Multiple FROM statements in one Dockerfile. Build stage compiles/builds the app (with all dev tools). Final stage copies only the artifact. Result: smaller image (no build tools in prod), smaller attack surface, faster deployment. Example: Go binary built in golang:1.21, copied to scratch image.

★★★★★ What is Docker Compose? When would you use it?

Tool for defining and running multi-container applications via YAML. Use it for local development and testing multi-service apps (web + db + cache), and for simple production deployments. Manages networking, volumes, dependencies, and startup order in one command.

★★★★★ What are the Docker networking drivers?

bridge (default, single-host), host (share host network), none (no networking), overlay (multi-host, Swarm/K8s), macvlan (direct network access). For production multi-container: user-defined bridge networks with DNS resolution by container name.

★★★★★ Difference between volumes, bind mounts, and tmpfs?

Named volumes: Docker-managed, recommended for production data persistence. Bind mounts: host directory mapped to container, great for dev (live code reload). tmpfs: in-memory, no persistence, for sensitive temp data. Use named volumes in production.

★★★★★ How do you run a container as a non-root user?

In Dockerfile: create a user (RUN useradd -r appuser) and switch with USER appuser. At runtime: docker run --user 1000:1000. Never run production containers as root — root in container can lead to root on host if container escapes.

★★★★★ What is the difference between docker stop and docker kill?

docker stop sends SIGTERM (graceful), waits for --stop-timeout (default 10s), then sends SIGKILL. docker kill sends SIGKILL immediately (or specified signal). Always use stop in production for graceful shutdown.

★★★★★ How do you pass environment variables to a container?

-e KEY=VALUE at runtime, --env-file file.env for multiple, ENV in Dockerfile (avoid for secrets), or in docker-compose.yml via environment: or env_file:. For secrets, use Docker secrets or AWS Secrets Manager.

★★★★★ What is docker exec? How is it different from docker run?

docker exec runs a command in an ALREADY RUNNING container. docker run creates and starts a NEW container. Use exec for debugging: docker exec -it bash. Use run to start new containers from images.

★★★★■ How do you limit container resources?

CPU: --cpus=0.5 (half core) or --cpu-shares (relative weight). Memory: --memory=512m, --memory-swap to control swap. In Compose: deploy.resources.limits. Without limits, a container can starve the host.

★★★★■ What is EXPOSE in a Dockerfile? Does it open ports?

EXPOSE is DOCUMENTATION only — it tells users which port the app uses. It does NOT open ports to the host. To actually publish ports, use -p hostPort:containerPort at runtime.

★★★★■ Difference between COPY and ADD in Dockerfile?

COPY simply copies files from host to image. ADD can also extract .tar archives and fetch remote URLs. Best practice: always prefer COPY unless you specifically need ADD's extra features.

★★★★■ What is docker system prune?

Removes all stopped containers, dangling images (no tag), unused networks, and optionally volumes. Safe for dev; be careful in prod. docker system df shows disk usage. docker image prune -a removes all unused images (including tagged but not referenced by containers).

★★★★■ How do you check docker logs in production?

docker logs --tail 200 -f . Configure log drivers for production: --log-driver=awslogs sends logs to CloudWatch. Set --log-opt max-size=10m max-file=3 to prevent disk fill. Use centralized logging (ELK, CloudWatch, Datadog) in production.

★★★★■ What is a Docker registry? Difference between Docker Hub and ECR?

Registry stores and distributes Docker images. Docker Hub is public registry. AWS ECR is private, IAM-authenticated, integrates with ECS/EKS, supports image scanning and lifecycle policies. Use ECR for all production images in AWS environments.

★★★★■ How does container DNS work?

On user-defined bridge networks, Docker runs a built-in DNS server. Containers resolve each other by container/service name. This does NOT work on the default bridge network. `docker run --name db postgres` → `docker run myapp` (can reach db by hostname "db" on same network).

★★★★■ What is docker inspect? What can you find there?

Returns JSON with full container/image configuration: network settings (IP, ports), environment variables, mount points, resource limits, health status, restart policy, labels, start/finish times. Essential for debugging.

★★★★■ What is Docker Swarm vs Kubernetes?

Docker Swarm: built-in Docker orchestration, simpler but less powerful. Kubernetes: industry-standard orchestration, more complex but far more features (auto-scaling, rolling updates, service mesh, etc.). In AWS: use ECS (simpler) or EKS (Kubernetes).

★★★★■ How do you build multi-architecture Docker images?

Use `docker buildx` with `--platform linux/amd64,linux/arm64`. Important for deploying to ARM-based instances (AWS Graviton). Example: `docker buildx build --platform linux/amd64,linux/arm64 -t myapp:latest --push`.

★★★★■ What is the difference between RUN, CMD, and ENTRYPOINT?

RUN executes during image BUILD (creates layer). CMD and ENTRYPOINT execute at CONTAINER STARTUP. CMD = default command/args, overridable. ENTRYPOINT = fixed executable, CMD becomes its arguments.

★★★★■ What is Docker Content Trust (DCT)?

DCT enforces image signing and verification. `DOCKER_CONTENT_TRUST=1` ensures you only pull/run signed images. Prevents supply chain attacks by ensuring image integrity. Use with Notary for image signing in enterprise.

★★★★■ What is a .dockerignore file?

Like `.gitignore` for Docker builds. Excludes files from the build context sent to the daemon. Always exclude: `.git`, `node_modules`, `.env`, `__pycache__`, `*.log`, test directories. Reduces build context size → faster builds.

★★★★■ What is the docker0 bridge interface?

`docker0` is the default virtual bridge network interface created by Docker on the Linux host. Default IP: `172.17.0.1`. All containers on the default bridge attach to it. User-defined networks create additional bridges (`docker_mynet` etc.).

★★★★■ What are dangling images?

Images with no tag and not referenced by any container — typically created when rebuilding without removing old images. Show with: `docker images -f dangling=true`. Remove with: `docker image prune`.

Top 20 Production Scenario Questions

■ Your container exits immediately after deployment. How do you diagnose?

`docker ps -a` → check status and `ExitCode`. `docker logs` → read error. `docker inspect` → check `ExitCode` and `OOMKilled` fields. Common causes: missing env var, wrong command, port conflict, missing file, out of memory.

■ Containers cannot communicate with each other. What do you check?

1) Are both on the same network? (`docker network inspect`) 2) Is it a user-defined network? (default bridge has no DNS) 3) `docker exec ping` to test DNS. 4) Check firewall/iptables rules. 5) Check service is actually listening (`netstat -tlnp` inside container).

■ Docker host disk is 95% full. What do you do?

`df -h /var/lib/docker`. `docker system df`. Safe cleanup: `docker system prune` (stops containers, dangling images). `docker image prune -a` (unused images). Check log files: `/var/lib/docker/containers/*/*...json.log` (configure max-size). Long term: set up ECR lifecycle policies, configure log rotation.

■ Container keeps restarting in a loop. How do you debug?

`docker inspect` → check `RestartCount` and `State`. `docker logs` → crash reason. `docker run --restart=no myimage` (override restart policy for debugging). Common cause: unhealthy healthcheck causing ECS/orchestrator to restart.

■ A new Docker image deployment is causing 502 errors. How do you roll back?

ECS: update service to previous task definition revision. Manual: `docker pull myimage:previous-tag` && `docker run...` CI/CD: trigger previous pipeline run or use `git revert` + redeploy. Always tag images with commit SHA or semantic versions — never rely only on `:latest`.

■ How do you update a running container with minimal downtime?

Never modify a running container. Deploy new version alongside. ECS: rolling update with minimum healthy percent. Docker Compose: `docker compose up --no-deps -d web` (update only web service). Swarm: `docker service update --image newimage myservice` (rolling update).

■ Container health checks are failing but the app seems OK. What do you check?

`docker inspect` → Health section → test command and output. Run healthcheck manually: `docker exec curl http://localhost/health`. Adjust `interval/timeout/retries/start_period`. Ensure health endpoint doesn't require auth. Check if tool (`curl`, `wget`) exists in image.

■ You suspect a container is consuming too much CPU. How do you investigate?

`docker stats` → live per-container CPU/MEM. `docker exec top` → processes inside container. `docker exec ps aux` → all processes. Set CPU limits: `--cpus=0.5` to throttle.

■ You need to copy a file from a stopped container. How?

`docker cp /path/file .local-path` works even on stopped containers. Or: `docker start` ; `docker exec cat /path/file`; `docker stop`

■ How do you inspect what environment variables a running container has?

`docker inspect` → Env field. `docker exec env` → prints all env vars from inside. Note: secrets passed as env vars are visible via inspect — use Docker secrets or secrets manager instead.

■ How would you debug a container that has no shell (distroless image)?

Option 1: `docker cp` to extract files. Option 2: `docker debug` (Docker Desktop feature — attaches debug shell). Option 3: Run a debug sidecar container sharing the same network/volume. Option 4: `kubectl debug` (in K8s) to add ephemeral container.

■ Your Docker build is slow. How do you optimize?

1) Check `.dockerignore` — exclude large dirs. 2) Reorder Dockerfile: copy dependency files (`package.json`, `requirements.txt`) before code. 3) Combine RUN commands with `&&`. 4) Use multi-stage build. 5) Use `--cache-from` in CI to reuse previous build cache. 6) Use BuildKit (`DOCKER_BUILDKIT=1`) for parallel builds.

■ How do you manage secrets in a Docker environment?

Dev: `.env` file with `docker-compose` (never commit). Production options: Docker Swarm secrets (`/run/secrets/name`), AWS Secrets Manager (app fetches at startup), AWS SSM Parameter Store, ECS task definition secrets from Secrets Manager. Never: ENV in Dockerfile, `-e` in `docker run` with literal values in CI logs.

■ How do you set up Docker logging for production?

Set log driver: `--log-driver=awslogs` (CloudWatch). Limit log size: `--log-opt max-size=10m --log-opt max-file=3` (json-file driver). ECS: configure `awslogs` driver in task definition. Avoid `syslog` to Docker host — use centralized logging (CloudWatch, ELK, Datadog).

■ Container networking: explain what happens when `docker run -p 8080:80` is executed.

Docker creates iptables NAT rules on the host: traffic to `host:8080` is forwarded to `container:80`. The container gets its own network namespace with `eth0` connected to `docker0` bridge. Host routes: `docker0` interface forwards to container IP. External clients → `host:8080` → iptables → `docker0` → `container:80`.

■ How do you back up and restore a Docker volume?

Backup: `docker run --rm -v myvolume:/data -v $(pwd):/backup ubuntu tar czf /backup/backup.tar.gz -C /data`. Restore: `docker run --rm -v myvolume:/data -v $(pwd):/backup ubuntu tar xzf /backup/backup.tar.gz -C /data`. For databases: always use the database's own dump/restore tools (`pg_dump`, `mysqldump`).

■ How do you deploy a Docker app to AWS ECS?

1) Build image. 2) Push to ECR. 3) Create/update ECS Task Definition with new image URI. 4) Update ECS Service to use new task definition. ECS handles rolling deployment. Automate with GitHub Actions/CodePipeline → CodeBuild → ECR → ECS deploy action.

■ Container IP changes every restart. How do services find each other?

Use Docker DNS — connect containers to a user-defined network and reference by container/service name. IPs are ephemeral and should never be hardcoded. In Compose, service names act as DNS hostnames. In ECS, use service discovery (Cloud Map) or ALB for inter-service communication.

■ How do you scan Docker images for vulnerabilities?

Tools: docker scout (built-in), trivy (open source, popular), AWS ECR image scanning (on push). Add to CI: `trivy image --exit-code 1 --severity HIGH,CRITICAL myimage`. Fails pipeline if critical vulns found. Also scan base images — use minimal bases (alpine, distroless).

■ What happens to container data when the container is removed?

All data in the writable container layer is permanently lost. Only data in mounted volumes persists. Always use named volumes or bind mounts for any data that must survive container restarts or replacements. Never rely on container filesystem for persistence.

DOCKER CHEAT SHEET — One-Page Revision

Quick Reference

Category	Commands
Run	<code>docker run -d -p 8080:80 --name web --restart unless-stopped nginx</code>
Logs	<code>docker logs -f --tail 100</code>
Exec	<code>docker exec -it bash</code>
Inspect	<code>docker inspect</code> <code>docker stats</code> <code>docker top</code>
Cleanup	<code>docker system prune</code> <code>docker system prune -a --volumes</code>
Build	<code>docker build -t myapp:1.0 .</code> <code>docker build --no-cache .</code>
Push/Pull	<code>docker push/pull repo/image:tag</code>
Networks	<code>docker network create mynet</code> <code>docker network inspect mynet</code>
Volumes	<code>docker volume create vol</code> <code>docker volume inspect vol</code>
Compose	<code>docker compose up -d --build</code> <code>docker compose down</code> <code>docker compose logs -f</code>

Key Concepts Summary

Concept	Remember This
Image vs Container	Image = template (read-only). Container = running instance (adds writable layer).
Layers	Each RUN/COPY/ADD creates a layer. Layers cached. Order least→most change.
CMD vs ENTRYPOINT	ENTRYPOINT = fixed exe. CMD = default args. Together: best pattern.
EXPOSE	Documentation only. Use -p to actually publish ports.
Bridge network DNS	User-defined networks only. Default bridge has no DNS.
Named volumes	Managed by Docker. Persists past container lifecycle. Use in prod.
Non-root user	Always. USER appuser in Dockerfile. Security requirement.
Multi-stage build	Build in fat image. Copy artifact to slim image. Smaller, safer.
Healthcheck	Always define. Compose depends_on condition: service_healthy needs it.
Secrets	Never in ENV in Dockerfile. Use Docker secrets or AWS Secrets Manager.

Interview Priority Matrix

Must Know (★★★★★)	Know Well (★★★★■)	Know Basics (★★★██)
Image vs Container	Multi-arch builds (buildx)	Docker Content Trust
Dockerfile instructions	Log drivers & CloudWatch	docker0 bridge internals
CMD vs ENTRYPOINT	ECR auth & lifecycle policies	ONBUILD instruction

Layer caching	ECS Task Definition	macvlan / overlay networks
Multi-stage builds	Compose depends_on + health	Docker Swarm basics
Networking (bridge, DNS)	Volume backup/restore	Namespaces & cgroups detail
Named volumes	OOMKill troubleshooting	Docker socket security
Non-root user	Resource limits (cpu/mem)	Volume drivers (EFS, etc.)
docker exec / logs / inspect	Image vulnerability scanning	docker save / load
docker system prune	CI/CD pipeline with ECR	LABEL metadata

■ SRE Interview Winning Answer Framework:

When answering Docker questions, follow: **What** → **Why** → **How** → **Production Example** →

Trade-offs

Example: "What is a named volume?" → Define it → Explain why (persistence beyond container lifecycle) → How to create/use it → Production: postgres data persists after container update → Trade-off: harder to back up than bind mounts but more portable.

Designed for Intermediate SRE / Cloud Operations Engineers (4 Years Experience)
DevOps Engineer • Platform Engineer • Infrastructure Engineer • Cloud Engineer